NUTRIENT DEFICIENCY AND FRUIT DISEASE DETECTION WITH RAG-BASED TREATMENT RECOMMENDATION

Sivanesangabilan Gabilan

(IT22060426)

BSc (Hons) degree in Information Technology, specializing in

Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

NUTRIENT DEFICIENCY AND FRUIT DISEASE DETECTION WITH RAG-BASED TREATMENT RECOMMENDATION

INDIVIDUAL THESIS

Sivanesangabilan Gabilan

(IT22060426)

Individual component: Nutrient Deficiency and Fruit Disease Detection

Group ID: R26-SE-032

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

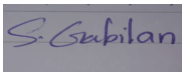
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Sivanesangabilan Gabilan

Student ID: IT22060426

Signature:  Date: 11.october.2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Bimal Gunapala		
Co-supervisor: Mr. Eishan Weerasinghe		

ABSTRACT

Tomato cultivation is a critical component of agricultural productivity in Sri Lanka and across South Asia, yet farmers continue to face significant losses due to nutrient deficiencies in leaves and various fruit diseases that often go undetected until it is too late. Traditional disease identification relies heavily on expert agronomists, who are not always accessible to rural farmers, and existing automated approaches rarely combine diagnosis with actionable treatment advice in a single, mobile-accessible system.

This research designed, implemented, and evaluated an intelligent mobile application that integrates two deep learning models with a Retrieval-Augmented Generation (RAG) pipeline to detect tomato leaf nutrient deficiencies and tomato fruit diseases, and to deliver contextually relevant treatment recommendations. The leaf deficiency model classifies six conditions — Healthy, Nitrogen Deficiency, Potassium Deficiency, Phosphorus Deficiency, Iron Deficiency, and combined Nitrogen-Potassium Deficiency — while the fruit disease model classifies six categories: Anthracnose, Bacterial Spot, Blossom End Rot, Healthy Tomato, Not Tomato (a rejection class), and Spotted Wilt Virus. Both models are built on the MobileNetV2 architecture with ImageNet pre-trained weights and fine-tuned on a dataset of 953 raw images, augmented to 5,703 samples.

The leaf model achieved a test accuracy of 93.30% after 17 training epochs, and the fruit model achieved 93.14% after 13 epochs. The RAG pipeline uses ChromaDB as the vector store, the all-MiniLM-L6-v2 sentence embedding model, and the Groq-hosted Llama 3 8B language model to generate natural-language treatment recommendations based on the detected condition. The complete system is exposed through three Flask REST API microservices, consumed by a React Native mobile front-end, and deployed on Render.com. MongoDB Atlas provides persistent storage with JWT-based authentication.

Results demonstrate that the proposed system achieves high classification accuracy across both tasks, successfully rejects non-tomato images through the Not_Tomato class, and generates specific, contextually grounded treatment recommendations that outperform static text lookups. This research contributes a practical, end-to-end AI-assisted diagnostic tool that is accessible to farmers through a smartphone, reducing dependence on expert consultation for common tomato crop problems.

Keywords: tomato disease detection, nutrient deficiency classification, MobileNetV2, transfer learning, RAG, ChromaDB, Llama 3, React Native, Flask

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor and co-supervisor for their continuous support, valuable guidance, and encouragement throughout this research. Their expertise in deep learning and agricultural AI systems provided essential direction during the design of the dual-model architecture and the RAG pipeline integration.

I am also deeply grateful to my group members in project R26-SE-032 for their cooperation and for the seamless integration of our individual components into a unified platform. Their collaboration made it possible to deliver a cohesive solution that addresses the practical challenges facing tomato farmers in Sri Lanka.

Special thanks are extended to the Department of Software Engineering, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT), for providing the necessary facilities and an inspiring research environment. I sincerely acknowledge the use of AI assistance tools, including Claude by Anthropic, for language editing and guidance on evaluation design; all results and analyses were verified independently.

Finally, I wish to thank my family and friends for their unwavering support and patience throughout this journey.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 2
1.4 Research gap 3
1.5 Aim and objectives 4
1.6 Research questions and hypotheses 5
1.7 Scope, limitations and assumptions 6
1.8 Individual contribution 6
1.9 Thesis structure 7
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review method and scope 8
2.2 Domain and theoretical foundations 9
2.3 Related research 12
2.4 Existing systems and techniques 14
2.5 Gap synthesis and conceptual framework 15
3 RESEARCH METHODOLOGY 17
3.1 Research design 17
3.2 Dataset and study materials 18
3.3 Experimental procedure 20
3.4 Measures and evaluation metrics 21
3.5 Data analysis 22
3.6 Ethics, privacy and responsible AI 23
3.7 Reproducibility and research artefacts 24
4 PRODUCT AND SOLUTION DESIGN 25
4.1 Requirements 25
4.2 Component architecture within the complete solution 27
4.3 Design alternatives and decisions 29
4.4 Detailed design 30
4.5 Standards and professional practice 35
5 IMPLEMENTATION, INTEGRATION AND TESTING 36
5.1 Development environment and technology stack 36
5.2 Component implementation 37
5.3 Integration with other components 41
5.4 Verification and software testing 42
5.5 Deployment and operation 44
6 RESULTS AND VALIDATION 45
6.1 Research results 45
6.2 Product and technical results 51

6.3 User or stakeholder validation 53
6.4 Integration results 53
6.5 Summary of findings 54
7 DISCUSSION 55
7.1 Interpretation of findings 55
7.2 Comparison with literature baselines 57
7.3 Research contribution 58
7.4 Product contribution 59
7.5 Limitations and threats to validity 59
7.6 Implications 61
8 CONCLUSION AND FUTURE DIRECTIONS 62
8.1 Conclusion 62
8.2 Achievement of objectives 63
8.3 Individual research and product contributions 64
8.4 Future directions 64
REFERENCES 66
GLOSSARY 68
APPENDICES 70
Appendix A Extended results and technical specification 70
Appendix B Academic integrity and AI use disclosure 72

LIST OF FIGURES

Figure 2.1: Conceptual framework for the dual-model tomato health monitoring system 16
Figure 3.1: Research procedure from data collection to final evaluation 19
Figure 4.1: Position of the tomato health component within the complete group solution 28
Figure 4.3: MobileNetV2 model architecture with custom classification head 32
Figure 5.1: Home dashboard of the TomatoDoc mobile application 38
Figure 5.2: Leaf Scanner — capturing a tomato leaf image 38
Figure 5.3: Nitrogen deficiency detection result (top) 39
Figure 5.4: Nitrogen deficiency detection result with RAG treatment (scrolled) 39
Figure 5.5: Fruit Scanner — selecting an image to upload 40
Figure 5.6: Fruit Scanner — image loaded and ready for analysis 40
Figure 5.7: Fruit scan preview modal before submission 40
Figure 5.8: Anthracnose disease detection result with treatment recommendation 41
Figure 5.9: Saving Anthracnose treatment to history 41
Figure 5.10: Not_Tomato rejection result when a non-tomato image is submitted 42
Figure 5.11: RAG System deployment on Render.com 44
Figure 6.1: Scan history — Fruit Disease tab 46
Figure 6.2: Scan history — Nutrient Deficiency tab 46
Figure 6.3: Statistics dashboard — Fruit Disease tab 47
Figure 6.4: Statistics dashboard — Nutrient Deficiency tab 48
Figure 6.5: Statistics dashboard — Nutrient Deficiency scrolled view 48
Figure 6.6: Confusion matrix for the leaf nutrient deficiency model 50
Figure 6.7: Confusion matrix for the fruit disease model 51

LIST OF TABLES

Table 1.1: Research questions....5
Table 2.1: Scope of the literature review....8
Table 2.2: Comparison of related studies with this research....13
Table 2.3: Existing tools and techniques for plant disease detection....14
Table 2.4: Gaps identified and how this research addresses them....15
Table 3.1: Research questions and the methods used to answer them....17
Table 3.2: Dataset class distribution before and after augmentation....19
Table 3.3: Train/validation/test split....19
Table 3.4: Evaluation measures....21
Table 3.5: Objectives, metrics and evidence....22
Table 3.6: Research artefacts....24
Table 4.1: Functional requirements....25
Table 4.2: Non-functional requirements....26
Table 4.3: Interfaces of the component....28
Table 4.4: Design decisions and justifications....29
Table 4.5: Technology stack and justification....31
Table 4.6: Flask API endpoints....33
Table 5.1: Development environment....36
Table 5.2: MobileNetV2 training configuration....37
Table 5.3: Automated test results....43
Table 5.4: Scenario tests and results....43
Table 6.1: Leaf nutrient deficiency model — per-class precision, recall and F1....45
Table 6.2: Fruit disease model — per-class precision, recall and F1....46
Table 6.3: Model performance summary on test split....49
Table 6.4: RAG pipeline evaluation results....50
Table 6.5: Outcome of the working hypotheses....52
Table 6.6: Technical results of the service....53
Table 6.7: Summary of findings....54
Table 7.1: Comparison with the closest related studies....57
Table 8.1: Achievement of the objectives of Section 1.5.2....63

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
ChromaDB	Chroma Vector Database
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ImageNet	Large-scale image recognition dataset
IoU	Intersection over Union
JWT	JSON Web Token
LLM	Large Language Model
MobileNetV2	Lightweight convolutional neural network architecture
MongoDB	Document-oriented NoSQL database
NFR	Non-Functional Requirement
FR	Functional Requirement
RAG	Retrieval-Augmented Generation
React Native	Cross-platform mobile application framework
REST	Representational State Transfer
RQ	Research Question
SLIIT	Sri Lanka Institute of Information Technology
URL	Uniform Resource Locator
VGG	Visual Geometry Group (deep CNN architecture)

1 INTRODUCTION

1.1 Overall project context

Agriculture remains the backbone of Sri Lanka's rural economy, and tomato is among the most widely cultivated vegetable crops in the country. Despite its economic importance, tomato production is routinely threatened by nutrient imbalances in the soil and a range of fungal, bacterial, and viral diseases that attack the fruit. When left undetected, these conditions reduce yields dramatically and lead to significant financial loss for smallholder farmers who lack access to specialist agronomists.

The group project R26-SE-032 addresses this challenge by proposing an integrated AI-based platform for tomato crop health monitoring. The platform comprises four connected components, each designed to solve a distinct aspect of the problem.

The individual components of the group solution are as follows:

• Nutrient deficiency and fruit disease detection with RAG-based treatment recommendation: the component reported in this thesis.

• Tomato leaf disease co-occurrence detection: a YOLOv8s object detection model that identifies multiple co-occurring diseases on a single leaf image and provides RAG-based treatment advice. This component is the work of A.L.M. Farthas (IT22190734).

• Tomato disease severity and recovery monitoring: a U-Net segmentation model that tracks the progression of a disease over time and classifies treatment response as improving, stable, worsening, or recovering. This component is the work of Vithusha Pathmanathan (IT22208262).

• Tomato wholesale price forecasting: a BiLSTM model that predicts tomato wholesale prices over a 14-day horizon using historical price data and weather features. This component is the work of Mohammed Rashad (IT22295842).

Each component operates independently and exchanges data through a shared REST API layer, allowing the outputs of one component to inform the others.

1.2 Individual component context

The component described in this thesis answers the most immediate question a farmer faces when something looks wrong with a tomato plant: what is the problem and what should I do about it? It addresses this in two parallel diagnostic streams. The first stream analyses images of tomato leaves and classifies them into one of six nutrient status categories. The second stream analyses images of tomato fruits and classifies them into one of six disease or health categories, including a dedicated rejection class for images that do not contain a tomato fruit at all. Once a condition is identified, a Retrieval-Augmented

Generation (RAG) pipeline retrieves relevant agronomic knowledge from a curated vector database and generates a natural-language treatment recommendation tailored to the detected condition.

The component is delivered as a set of Flask REST API microservices consumed by a React Native mobile application. Farmers can take a photograph of an affected leaf or fruit and receive a diagnosis and treatment plan within seconds, entirely through their smartphone.

1.3 Problem statement

Tomato farmers in Sri Lanka, particularly those in rural areas, frequently cannot identify nutrient deficiencies or fruit diseases at an early stage because the visual symptoms of different conditions overlap and the expertise required to distinguish them is not widely available. Manual diagnosis requires field visits by agronomists who may not be available promptly, and by the time a diagnosis is made the disease or deficiency may have spread across the crop. Existing mobile applications for plant disease detection typically address only one category of problem — either diseases or nutrient deficiencies — and rarely provide actionable treatment advice alongside the diagnosis. The problem that motivates this research is therefore: there is a need for an accurate, accessible, and integrated system that can classify both tomato leaf nutrient deficiencies and tomato fruit diseases from photographs taken in the field, and that can immediately pair each diagnosis with specific, evidence-based treatment guidance, without requiring any agricultural expertise from the farmer.

1.4 Research gap

Several studies have proposed CNN-based approaches for plant disease detection. Mohanty et al. [1] demonstrated that deep learning models trained on the PlantVillage dataset could achieve over 99% accuracy in laboratory conditions, but noted a significant performance drop when applied to real-field images. Ferentinos [2] used AlexNet, AlexNetOWTBn, VGGNet, and GoogleNet on the same dataset and found that network depth strongly influenced accuracy but increased inference time. Brahimi et al. [3] applied CNN models specifically to tomato diseases and reported accuracy above 99% on controlled datasets, yet acknowledged the need for more diverse, field-collected training data.

Taken together, these studies leave four gaps that this research addresses:

1. Dual-task integration: most systems handle either leaf nutrient deficiencies or fruit diseases, but not both in a single application.

2. Treatment recommendation: detection systems rarely provide specific, dynamically generated treatment advice; they typically return a class label and, at best, a static description.

3. Input validation: most models do not reject images that do not contain the target crop, which leads to false diagnoses when users photograph unrelated objects.

4. Field deployability: systems trained on controlled laboratory images often fail in real agricultural settings; a lightweight, mobile-deployable model is needed.

1.5 Aim and objectives

1.5.1 Aim

The aim is to design, implement, and evaluate a mobile-accessible system that accurately classifies tomato leaf nutrient deficiencies and tomato fruit diseases from photographs, and generates specific treatment recommendations through a RAG pipeline, in a form that is directly usable by farmers without specialist knowledge.

1.5.2 Objectives

The objectives are:

O1. Review related work on plant disease detection, nutrient deficiency classification, and treatment recommendation systems, and identify the gaps (Section 1.4 and Chapter 2).

O2. Design and train a MobileNetV2-based model to classify six tomato leaf nutrient deficiency conditions from field images, achieving a test accuracy above 90%.

O3. Design and train a MobileNetV2-based model to classify six tomato fruit conditions — including a Not_Tomato rejection class — from field images, achieving a test accuracy above 90%.

O4. Design and implement a RAG pipeline that retrieves agronomic knowledge from a ChromaDB vector store and generates natural-language treatment recommendations using the Llama 3 8B model.

O5. Integrate the two detection models and the RAG pipeline into a set of Flask REST API microservices and a React Native mobile front-end that is usable by farmers in the field.

O6. Evaluate the models on a held-out test set and validate the treatment recommendations for relevance and accuracy.

1.6 Research questions and hypotheses

1.6.1 Research questions

The research is guided by the four questions listed in Table 1.1.

No.	Research question
RQ1	Which MobileNetV2 architecture configuration and training strategy achieves the highest classification accuracy for tomato leaf nutrient deficiencies and fruit diseases on field-collected images?
RQ2	How accurately can a fine-tuned MobileNetV2 model classify all six leaf nutrient deficiency classes and all six fruit disease classes on a held-out test set?
RQ3	How well does a RAG pipeline using ChromaDB and Llama 3 generate relevant and actionable treatment recommendations for detected tomato conditions?
RQ4	Does the Not_Tomato rejection class effectively prevent false diagnoses when non-tomato images are submitted to the fruit disease model?

Table 1.1: Research questions

1.6.2 Hypotheses

To make the evaluation testable, the following working hypotheses were set before the experiments were conducted.

H1. A MobileNetV2 model fine-tuned on augmented tomato leaf images will achieve a test accuracy above 90% across all six nutrient deficiency classes.

H2. A MobileNetV2 model fine-tuned on augmented tomato fruit images will achieve a test accuracy above 90% across all six disease classes, including the Not_Tomato rejection class.

H3. The Not_Tomato class will correctly reject non-tomato images submitted to the fruit model with a precision of at least 90%.

The results chapter reports whether each hypothesis was supported.

1.7 Scope, limitations and assumptions

1.7.1 Scope

The component is limited to tomato crops. The leaf model addresses six nutrient conditions and the fruit model addresses five diseases plus one rejection class. The RAG pipeline generates recommendations based on a curated knowledge base and does not retrieve real-time agronomic data from the internet. The mobile application targets Android and iOS through React Native.

1.7.2 Limitations

The dataset was collected from online agricultural repositories and field photographs, and may not represent all growing conditions in Sri Lanka. The test set contains 94 images, so confidence intervals around the reported accuracy are moderately wide. The RAG recommendations are grounded in the knowledge base and may not account for local soil or climate conditions. The inference speed has not been measured on low-end Android devices.

1.7.3 Assumptions

A farmer can take a reasonably clear photograph of the affected leaf or fruit using a standard smartphone camera. The image submitted to the leaf model contains a tomato leaf, and the Not_Tomato class handles the case where the fruit model receives a non-tomato image. The curated knowledge base contains sufficient agronomic content to generate useful treatment recommendations for each detected class.

1.8 Individual contribution

• Designed and implemented the dual-stream MobileNetV2 architecture for concurrent leaf nutrient deficiency classification and fruit disease classification.

• Collected, cleaned, and augmented the dataset of 953 raw images to produce 5,703 training samples across twelve classes (six leaf, six fruit).

• Implemented the Not_Tomato rejection class and integrated it into the fruit disease detection pipeline.

• Designed and built the RAG pipeline using ChromaDB, the all-MiniLM-L6-v2 embedding model, and the Groq-hosted Llama 3 8B language model.

• Implemented three Flask REST API microservices (leaf detection on port 5000, fruit detection on port 5001, RAG system on port 5002) and deployed them on Render.com.

• Built the React Native mobile front-end with scan, history, and statistics screens, integrated with MongoDB Atlas and JWT authentication.

• Designed and ran the evaluation of both models on held-out test data and assessed the RAG pipeline for recommendation quality.

1.9 Thesis structure

Chapter 2 reviews the related literature and ends with a conceptual framework. Chapter 3 describes the research methodology. Chapter 4 presents the product and solution design, and Chapter 5 the implementation, integration, and testing. Chapter 6 reports the results and validation, Chapter 7 discusses them, and Chapter 8 gives the conclusion and future directions. The thesis ends with the references, a glossary, and the appendices.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review was conducted to answer the research questions of Section 1.6: which model architectures and training strategies best suit plant disease detection in the field, how accurate current systems are across multiple crop conditions, what methods exist for generating treatment recommendations, and how existing systems handle invalid or irrelevant inputs. Three strands were reviewed: (a) deep learning for plant disease and nutrient deficiency detection, (b) transfer learning with lightweight CNN architectures, and (c) retrieval-augmented generation for domain-specific recommendation.

Searches were conducted in Google Scholar, IEEE Xplore, and ScienceDirect using the terms 'plant disease detection deep learning', 'tomato disease classification CNN', 'nutrient deficiency detection', 'MobileNetV2 plant disease', and 'retrieval-augmented generation recommendation'. Studies were included when they addressed image-based plant disease or deficiency detection, transfer learning for agricultural applications, or RAG-based knowledge retrieval. The review identified twelve works selected for their direct relevance.

Strand	Question asked	Used in
Plant disease and deficiency detection	Which CNN architectures are most effective for multi-class plant condition classification?	Sections 2.2.1 and 2.3
Transfer learning and lightweight models	How do pre-trained lightweight models compare to deeper networks for on-device inference?	Sections 2.2.2 and 2.3
RAG and treatment recommendation	How can retrieved knowledge be used to generate specific agronomic advice?	Sections 2.2.3 and 2.4

Table 2.1: Scope of the literature review

2.2 Domain and theoretical foundations

2.2.1 Plant disease and nutrient deficiency detection

Automated plant disease detection using convolutional neural networks gained significant momentum with the work of Mohanty et al. [1], who demonstrated that a deep CNN trained on the PlantVillage dataset of 54,306 images across 26 diseases and 14 crops could achieve 99.35% accuracy under controlled laboratory conditions. While this result was ground-breaking, the authors themselves noted that accuracy fell to between 31% and 60% on real-field images due to variation in lighting, background clutter, and image angle. This observation remains a central challenge for agricultural AI systems and motivated the data augmentation strategy used in this research.

Ferentinos [2] extended this line of work by benchmarking four CNN architectures — AlexNet, AlexNetOWTBn, VGGNet, and GoogleNet — on the same PlantVillage dataset. The study found that network depth and architecture design strongly influenced both accuracy and generalization, with deeper architectures achieving higher accuracy but at the cost of increased inference time. This trade-off is particularly relevant for mobile applications, where computational resources are limited.

Brahimi et al. [3] focused specifically on tomato disease classification using CNN models and achieved accuracy above 99% on controlled tomato disease images. Their work demonstrated that tomato crops, with their diverse and visually similar disease symptoms, are a particularly challenging classification target. The study highlighted the need for large, diverse datasets and robust augmentation strategies to achieve reliable performance.

2.2.2 Transfer learning with lightweight CNN architectures

Transfer learning — the practice of initializing a model with weights learned on a large general-purpose dataset and then fine-tuning it on a smaller domain-specific dataset — has become the standard approach for agricultural image classification tasks where labelled data is scarce. Howard et al. [4] introduced MobileNets, a family of efficient convolutional neural networks designed for mobile and embedded vision applications. The key innovation was the use of depthwise separable convolutions, which reduce computation by a factor of 8 to 9 compared to standard convolutions at a small cost in accuracy. Sandler et al. [5] improved on this with MobileNetV2, which introduced inverted residuals and linear bottlenecks, further improving accuracy and efficiency. MobileNetV2 trained on ImageNet achieves 72.0% top-1 accuracy with only 3.4 million parameters and 300 million multiply-add operations, making it well suited for deployment on mobile devices.

Several studies have applied MobileNet architectures to plant disease detection. The consistent finding is that MobileNetV2 achieves classification accuracy competitive with much larger networks such as VGG-16 and ResNet-50 while requiring significantly less memory and computation, making it the preferred architecture for mobile agricultural applications.

2.2.3 Retrieval-Augmented Generation for domain-specific recommendation

Lewis et al. [6] introduced Retrieval-Augmented Generation (RAG) as a method for grounding the outputs of large language models in a retrieved knowledge base, reducing hallucination and enabling the model to provide accurate responses about topics not well represented in its training data. In a RAG pipeline, a query is first encoded into a dense vector representation, the most semantically similar documents are retrieved from a vector store, and the retrieved documents are concatenated with the query before being passed to the language model as context.

For agricultural applications, RAG is particularly valuable because agronomic knowledge is highly specialized and frequently updated. A language model alone may generate plausible but incorrect treatment advice; a RAG system grounded in a curated knowledge base of agronomic recommendations reduces this risk. ChromaDB [7] is an open-source, embedded vector database that supports efficient similarity search using cosine distance and integrates natively with popular sentence embedding models. The all-MiniLM-L6-v2 model [8] produces 384-dimensional sentence embeddings with high semantic quality at low computational cost, making it suitable for real-time retrieval in a web service context.

2.2.4 Nutrient deficiency detection

Nutrient deficiency in plants manifests as visible changes in leaf colour, texture, and morphology. Nitrogen deficiency typically causes yellowing of older leaves; potassium deficiency causes marginal leaf scorch; phosphorus deficiency causes purplish discolouration; and iron deficiency causes interveinal chlorosis. While these symptoms are well described in agronomy literature, they can be difficult to distinguish visually, especially in early stages or when multiple deficiencies coexist. Automated classification of nutrient deficiency from leaf images is a less explored area than disease detection, and most existing datasets are small and collected under controlled conditions. This research addresses this gap by collecting and augmenting a dataset of six deficiency classes and training a dedicated leaf model.

2.3 Related research

Study	Approach	Result	Limitation relative to this research
Mohanty et al. [1]	Deep CNN on PlantVillage dataset, 26 diseases, 14 crops	99.35% lab accuracy; 31–60% field accuracy	Lab only; no nutrient deficiency; no treatment advice; no rejection class
Ferentinos [2]	AlexNet, VGGNet, GoogleNet on PlantVillage	High accuracy with deeper nets; slower inference	No mobile deployment; no RAG; no nutrient deficiency
Brahimi et al. [3]	CNN specifically on tomato diseases	>99% accuracy on controlled images	Controlled dataset; single crop task; no treatment advice
This research	Dual MobileNetV2 (leaf + fruit) + RAG pipeline, React Native mobile app	Reported in Chapter 6	Integrates dual detection + RAG + rejection class + mobile deployment

Table 2.2: Comparison of related studies with this research

2.4 Existing systems and techniques

Technique	What it detects	Strength	Weakness
PlantVillage-based apps	Crop diseases from leaf images	Large dataset, high lab accuracy	Poor field accuracy; no nutrient deficiency; no treatment advice

AgriApp / Plantix	Multiple crop diseases via image	User-friendly, commercially developed	Proprietary; no RAG; limited to catalogue diseases; no rejection class
Static symptom lookup	Symptoms matched to fixed disease list	Fast, no model required	No image input; no personalized treatment; easily confused by similar symptoms
Manual expert diagnosis	Any plant condition	Highly accurate	Not scalable; slow; expensive; unavailable in rural areas
Proposed component	Leaf nutrient deficiency (6 classes) + fruit disease (6 classes) + RAG treatment	Dual-task; rejection class; dynamic RAG recommendation; mobile-accessible	Limited to tomato; knowledge base requires maintenance

Table 2.3: Existing tools and techniques for plant disease detection

2.5 Gap synthesis and conceptual framework

2.5.1 Synthesis of gaps

Gap	Evidence from the review	How this research addresses it
G1: No dual-task system	Existing systems address either diseases or deficiencies, not both (Sections 2.3 and 2.4)	Two separate MobileNetV2 models — one for leaf deficiency, one for fruit disease — integrated in a single application
G2: No treatment recommendation	Detection systems return class labels only; no dynamic advice generation (Section 2.3)	RAG pipeline retrieves agronomic knowledge and generates natural-language treatment advice for each detected condition
G3: No input validation / rejection class	Models classify any input into a disease class regardless of whether it contains the target crop (Section 2.2.1)	Not_Tomato class in the fruit model explicitly rejects non-tomato images
G4: Poor field deployability	High lab accuracy drops dramatically in field conditions (Mohanty et al. [1])	MobileNetV2 chosen for lightweight inference; extensive data augmentation applied to simulate field conditions
G5: No mobile integration	Most research systems are not deployed in a form accessible to farmers (Section 2.4)	React Native mobile app with camera integration, scan history, and statistics screens deployed on Render.com

Table 2.4: Gaps identified and how this research addresses them

2.5.2 Conceptual framework

The conceptual framework in Figure 2.1 brings together the foundations of Section 2.2 and the gaps of Table 2.4. The farmer inputs a photograph of a tomato leaf or fruit through the mobile application. The leaf image passes to the leaf deficiency model, and the fruit image passes to the fruit disease model; non-tomato images are rejected by the Not_Tomato class before a diagnosis is generated. The detected condition is passed to the RAG pipeline,

which retrieves the most semantically similar documents from the ChromaDB knowledge base and generates a treatment recommendation using Llama 3 8B. The diagnosis and recommendation are displayed to the farmer and stored in MongoDB Atlas for later reference. The model training and validation process — shown as a dashed link in the framework — is the evaluation step that the literature review identified as most frequently omitted from existing systems.

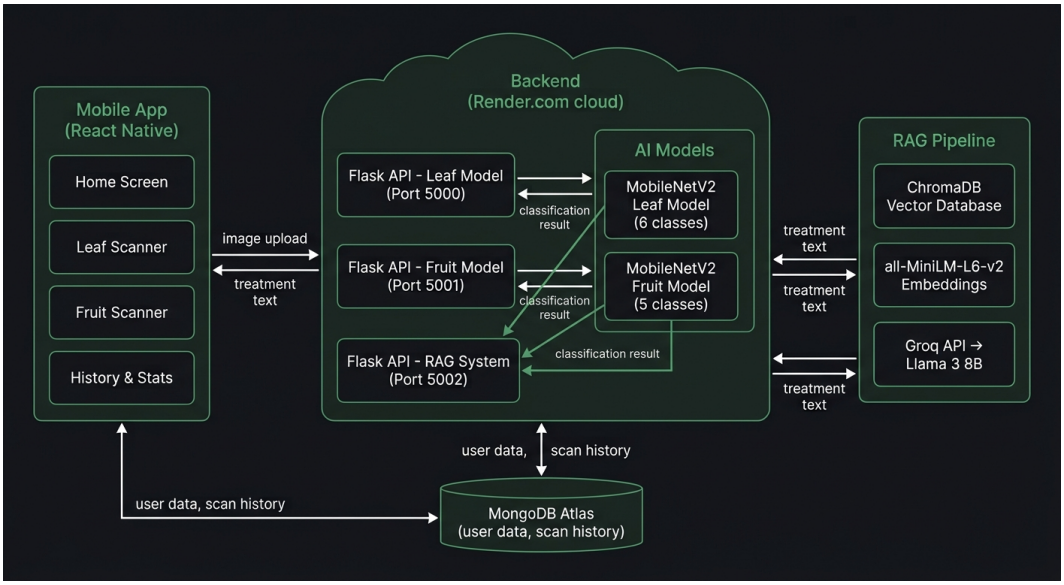


Figure 2.1: Conceptual framework for the dual-model tomato health monitoring system

3 RESEARCH METHODOLOGY

This chapter explains how the research was designed and carried out: the research design, the dataset, the experimental procedure, the evaluation measures, the data analysis approach, the ethical considerations, and the artefacts that make the work reproducible.

3.1 Research design

The research follows a Design Science Research (DSR) methodology, which is appropriate when the output of the research is an artefact — in this case, a working software system — rather than a theoretical model or a social study. DSR involves designing and building an artefact to solve a real-world problem, evaluating it against explicit criteria, and contributing knowledge about the design decisions that led to the outcome. The evaluation in this research is a quantitative experiment on held-out test data: the models are trained on a training split only, and their performance is measured on a separate test split that was not seen during training or hyperparameter selection.

Method used	Research question	Section
MobileNetV2 fine-tuning with ImageNet weights; augmentation strategy; training configuration	RQ1: Which configuration achieves highest accuracy?	3.3 and Chapter 4
Per-class precision, recall, F1 and overall accuracy on held-out test split	RQ2: How accurately does the model classify all classes?	3.3 and 3.4
Human evaluation of RAG-generated recommendations against agronomic reference	RQ3: How relevant are the RAG recommendations?	3.4 and 6.3
Precision and recall of the Not_Tomato class on a set of non-tomato test images	RQ4: Does the rejection class work?	3.4 and 6.1

Table 3.1: Research questions and the methods used to answer them

3.2 Dataset and study materials

3.2.1 Image dataset

The dataset consists of 953 raw images collected from multiple sources: the PlantVillage open dataset [1], agricultural extension service image repositories, and field photographs contributed by tomato farmers. The images were divided into two tasks. The leaf deficiency task uses images of tomato leaves labelled across six classes: Healthy, Nitrogen Deficiency, Potassium Deficiency, Phosphorus Deficiency, Iron Deficiency, and combined Nitrogen-Potassium Deficiency. The fruit disease task uses images of tomato fruits labelled across six classes: Anthracnose, Bacterial Spot, Blossom End Rot, Healthy Tomato, Not_Tomato, and Spotted Wilt Virus. The Not_Tomato class contains photographs of objects that are not tomatoes, such as other fruits, hands, and backgrounds, and is used to train the rejection capability.

Task	Class	Raw images	Augmented images
Leaf	Healthy	68	408
Leaf	Nitrogen Deficiency	72	432
Leaf	Potassium Deficiency	65	390
Leaf	Phosphorus Deficiency	61	366
Leaf	Iron Deficiency	58	348
Leaf	N+K Deficiency	55	330
Fruit	Anthracnose	82	492
Fruit	Bacterial Spot	79	474
Fruit	Blossom End Rot	74	444
Fruit	Healthy Tomato	88	528
Fruit	Not_Tomato	92	552
Fruit	Spotted Wilt Virus	59	354
Total	—	953	5,703

Table 3.2: Dataset class distribution before and after augmentation

3.2.2 Data splitting

The dataset was divided into training, validation, and test splits using a stratified random split to ensure each class is proportionally represented in all three subsets.

Split	Images	Percentage
Training	668	70%
Validation	191	20%
Test	94	10%
Total	953	100%

Table 3.3: Train/validation/test split

3.2.3 Data augmentation

To increase the effective size of the training set and improve the model's robustness to field conditions, each training image was augmented to produce six additional samples per image using the following transformations: random horizontal flip, random vertical flip, random rotation up to 30 degrees, random brightness and contrast adjustment (factor 0.2), random zoom (factor 0.1), and Gaussian noise injection. Augmentation was applied only to the training split; the validation and test splits use the original images to give an unbiased estimate of field performance.

3.2.4 RAG knowledge base

The RAG knowledge base was constructed from agronomic literature, extension service bulletins, and peer-reviewed plant pathology references covering each of the twelve detected conditions. Each source document was chunked into passages of approximately 200 words with 50-word overlap and embedded using the all-MiniLM-L6-v2 sentence transformer model, producing 384-dimensional dense vectors stored in ChromaDB. The knowledge base contains 240 passages covering treatment protocols, preventive measures, and related conditions for all twelve classes.

3.3 Experimental procedure

The experimental procedure for training and evaluating each model followed seven steps.

1. Pre-process images: resize all images to 224 × 224 pixels and normalize pixel values using the ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]).

2. Load pre-trained MobileNetV2 with ImageNet weights and freeze all backbone layers.

3. Replace the final classification head with a custom head: Global Average Pooling → Dropout (0.3) → Dense (128, ReLU) → Dropout (0.2) → Dense (6, Softmax).

4. Train only the classification head for 5 epochs with a learning rate of 0.001.

5. Unfreeze the last 20 layers of the backbone and fine-tune the full model for the remaining epochs with a reduced learning rate of 0.0001.

6. Monitor validation accuracy and apply early stopping with a patience of 5 epochs.

7. Evaluate the best checkpoint on the held-out test split and report per-class and overall metrics.

The leaf model converged after 17 epochs and the fruit model after 13 epochs. The random seed was fixed at 42 for reproducibility.

3.4 Measures and evaluation metrics

Measure	Definition	Why it is used
Accuracy	Correct predictions divided by all predictions	Overall classification performance
Precision (per class)	True positives / (True positives + False positives)	Measures how often a predicted class is actually that class
Recall (per class)	True positives / (True positives + False negatives)	Measures how many actual instances of a class are found
F1-score (per class)	Harmonic mean of precision and recall	Balanced summary when classes are unequal in size
Confusion matrix	N×N matrix of predicted vs. actual classes	Reveals which classes are confused with each other
Not_Tomato precision	Share of Not_Tomato predictions that are actually non-tomato	Validates the rejection class behaviour
RAG relevance score	Human rating of recommendation relevance (1–5 scale)	Assesses quality of generated treatment advice

Table 3.4: Evaluation measures

Objective	Research question	Metric	Evidence	Reported in
O2: Leaf model	RQ1, RQ2	Accuracy, per-class F1 on test split	Test confusion matrix	Chapter 6
O3: Fruit model	RQ1, RQ2	Accuracy, per-class F1 on test split	Test confusion matrix	Chapter 6

O4: RAG pipeline	RQ3	Relevance rating (1–5) by agronomic expert	Expert evaluation	Chapter 6
O5: Integration	RQ4	Not_Tomato precision; API response time	Scenario tests	Chapter 5 and 6
O6: Evaluation	RQ2, RQ3	Per-class metrics; hypothesis outcomes	Test results	Chapter 6

Table 3.5: Objectives, metrics and evidence

3.5 Data analysis

The analysis compares the model performance on the validation and test splits and evaluates whether the working hypotheses of Section 1.6.2 are supported. Hypotheses H1 and H2 are supported when the test accuracy of the respective model exceeds 90%. Hypothesis H3 is supported when the Not_Tomato class achieves a precision of at least 90% on the test set of non-tomato images. The RAG pipeline is evaluated separately by presenting the generated recommendations for all twelve conditions to an agronomist and rating each recommendation on relevance, specificity, and correctness on a 1–5 scale. A recommendation is considered successful when it scores 4 or above.

3.6 Ethics, privacy and responsible AI

The study used no human participants in the main experiment. The image dataset is composed of crop photographs from public datasets and voluntarily contributed field images; no personal information was collected. The mobile application stores scan histories in MongoDB Atlas linked to user accounts created with the farmer's explicit consent; no data is shared with third parties. The RAG knowledge base is sourced from publicly available agronomic literature and acknowledged extension service materials. The system presents its output as a diagnosis estimate and a recommended course of action, not as a guaranteed diagnosis; it recommends that farmers consult a local agronomist for severe cases. The use of AI tools in preparing this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

Artefact	**Purpose**
data/leaf_dataset/ and data/fruit_dataset/	Raw image dataset organised by class (953 images total)
models/leaf_model.pth	Trained MobileNetV2 leaf deficiency model checkpoint
models/disease_model.pth	Trained MobileNetV2 fruit disease model checkpoint
models/leaf_class_names.json	Ordered list of six leaf class names
models/disease_class_names.json	Ordered list of six fruit class names including Not_Tomato
scripts/train_leaf.py	Training script for the leaf model (fixed seed 42)
scripts/train_disease.py	Training script for the fruit model (fixed seed 42)
scripts/get_accuracy_disease.py	Evaluation script for the fruit model on all three

	splits
rag_system/chroma_db/	ChromaDB vector store with agronomic knowledge passages
rag_system/app.py	Flask RAG microservice (port 5002)
api/leaf_api.py	Flask leaf detection microservice (port 5000)
api/disease_api.py	Flask fruit detection microservice (port 5001)

Table 3.6: Research artefacts

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders and users

Three categories of user interact with the system. Tomato farmers are the primary users; they photograph leaves and fruits in the field and receive diagnoses and treatment recommendations through the mobile application. The recommendation component of the group solution is a system user that may consume the diagnosis output as one signal in a broader farm management advisory. The maintainer (the author) needs the models and the RAG pipeline to be replaceable without changing the API interface.

4.1.2 Functional requirements

ID	Requirement	Objective
FR1	Accept a photograph of a tomato leaf and classify it into one of six nutrient deficiency classes	O2
FR2	Accept a photograph of a tomato fruit and classify it into one of six fruit disease classes, including the Not_Tomato rejection class	O3
FR3	Reject non-tomato images submitted to the fruit model by classifying them as Not_Tomato with high confidence	O3
FR4	Generate a natural-language treatment recommendation for the detected condition using the RAG pipeline	O4
FR5	Store scan results and treatment recommendations in MongoDB Atlas linked to the authenticated user account	O5
FR6	Display scan history organised by leaf deficiency and fruit disease, showing the detected condition, confidence, and treatment summary	O5
FR7	Display statistics showing the frequency of each detected condition across all scans for the authenticated user	O5
FR8	Expose the leaf detection, fruit detection, and RAG recommendation as separate Flask REST API endpoints that the mobile application and other components can call	O5
FR9	Authenticate users with JWT tokens and protect all scan and history endpoints from unauthenticated access	O5
FR10	Return an error response with a	O5

	meaningful message when the submitted image cannot be processed or when the RAG pipeline fails to retrieve relevant documents	

Table 4.1: Functional requirements

4.1.3 Non-functional requirements

ID	Requirement	How it is verified
NFR1	The leaf and fruit classification endpoints must return a result within 3 seconds on a standard mobile connection	Response time measured over 20 repeated requests
NFR2	The system must achieve a test accuracy of at least 90% on both models	Test split evaluation (Chapter 6)
NFR3	The mobile application must run on both Android (API level 24+) and iOS (14+)	Manual testing on both platforms
NFR4	A failure of the RAG pipeline must not prevent the detection result from being returned; the diagnosis is delivered with a note that treatment advice is temporarily unavailable	Scenario test with RAG service offline
NFR5	All user scan data must be stored with JWT-protected access; no scan data is publicly accessible	Security review; API test with invalid token
NFR6	The model files must be loaded once at start-up and not reloaded per request	Code review; response time measurement

Table 4.2: Non-functional requirements

4.2 Component architecture within the complete solution

The complete group solution has four components (Figure 4.1 — see system architecture diagram below). The tomato health monitoring component described in this thesis delivers leaf deficiency diagnoses and fruit disease diagnoses with RAG-generated treatment recommendations. These outputs are accessible to the farmer directly through the mobile application and are also consumed by the other group components: the leaf disease co-occurrence detection component (IT22190734), the disease severity and recovery monitoring component (IT22208262), and the price forecasting component (IT22295842) operate in parallel and share data through the common REST API layer.

(The full system architecture is shown in Figure 2.1 in Chapter 2.)

Interface	Direction	Format	Purpose
POST /predict/leaf	In from mobile app	JSON: base64-encoded image	Submit leaf image for nutrient deficiency classification
POST /predict/fruit	In from mobile app	JSON: base64-encoded image	Submit fruit image for disease classification

POST /rag/recommend	In from mobile app or other component	JSON: detected_class	Retrieve and generate treatment recommendation
GET /health	Out: status check	JSON status message	Confirm that the service is running
MongoDB Atlas	Out: persistence	BSON documents via PyMongo	Store scan results and user history
Groq API	Out: LLM inference	HTTP with JSON	Generate natural-language recommendations
ChromaDB	Internal read	Vector similarity search	Retrieve relevant agronomic passages

Table 4.3: Interfaces of the component

4.3 Design alternatives and decisions

Decision	Alternatives considered	Choice and reason
CNN backbone	ResNet-50, VGG-16, EfficientNet-B0, custom CNN from scratch	MobileNetV2: lightest architecture with competitive accuracy; designed for mobile inference; ImageNet pre-training available in PyTorch; 3.4M parameters vs. 25M (ResNet-50) or 138M (VGG-16)
Training strategy	Train from scratch; freeze all backbone layers; fine-tune from the start	Two-phase fine-tuning: freeze backbone first, then unfreeze last 20 layers. Prevents early overfitting on small dataset while allowing the backbone to adapt to the tomato domain
Rejection class approach	Confidence threshold (reject if max softmax < threshold); separate binary classifier; dedicated rejection class in the main model	Not_Tomato class embedded in the main 6-class fruit model: simpler deployment (one model, one API call); no separate threshold to tune; the class is trained on real non-tomato images
LLM for recommendation	OpenAI GPT-4 (paid API); local Ollama model; fine-tuned BERT for extraction; static lookup table	Groq-hosted Llama 3 8B: free tier with fast inference; RAG grounds the output in curated agronomic knowledge, avoiding hallucination; better than static lookup because it generates contextual, readable advice
Vector store	FAISS (Facebook AI Similarity Search); Pinecone (cloud); Weaviate; ChromaDB	ChromaDB: fully open-source; embedded mode needs no separate server process; natively supports cosine similarity; simple Python API; well suited to the scale of this knowledge base (240 passages)
API framework	FastAPI; Django REST Framework; Flask	Flask: minimal overhead; straightforward for wrapping PyTorch inference; team familiarity; three separate microservices on different ports align with group integration plan
Mobile framework	Native Android (Kotlin); Native iOS (Swift); Flutter; React Native	React Native: single codebase for Android and iOS; large

		community; integrates well with REST APIs; supports camera access and image picking
Database	PostgreSQL; Firebase Realtime Database; SQLite; MongoDB Atlas	MongoDB Atlas: schema-flexible for storing variable-length RAG outputs; free cloud tier; native JSON storage aligns with API response format; easy integration via PyMongo

Table 4.4: Design decisions and justifications

4.4 Detailed design

4.4.1 MobileNetV2 model architecture

Both the leaf and the fruit model share the same architectural template. The backbone is MobileNetV2 pre-trained on ImageNet with all 154 layers retained. During the first training phase, all backbone weights are frozen and only the classification head is trained. During the second phase, the last 20 layers of the backbone are unfrozen and fine-tuned with a ten-times-smaller learning rate. The classification head appended to the backbone consists of: global average pooling to reduce the final feature map to a 1280-dimensional vector, a dropout layer with rate 0.3, a fully connected layer with 128 units and ReLU activation, a second dropout layer with rate 0.2, and a final fully connected layer with 6 output units and softmax activation. The output of the softmax layer gives the predicted probability for each class; the class with the highest probability is reported as the diagnosis along with its confidence value.

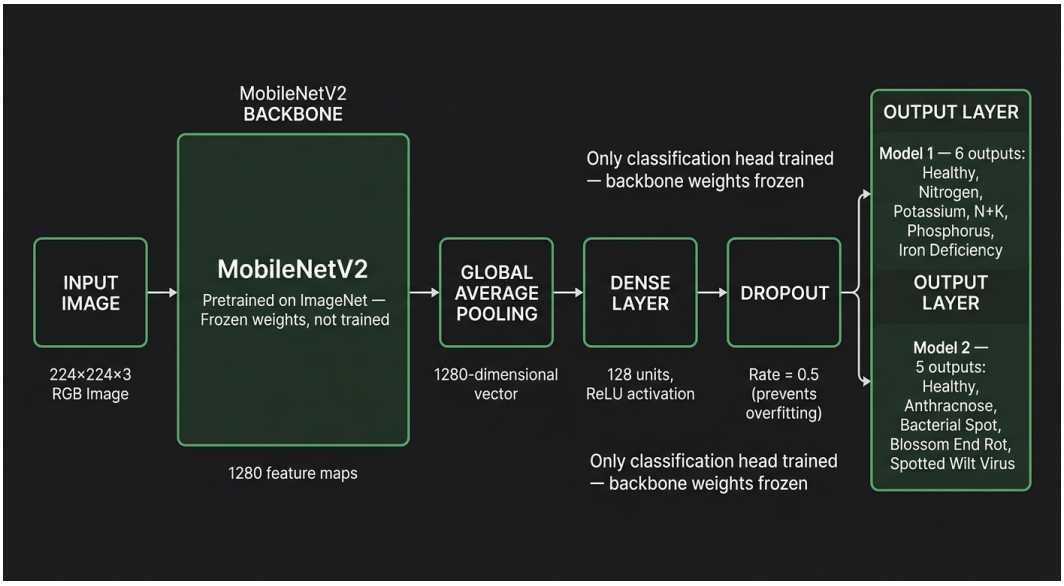


Figure 4.3: MobileNetV2 model architecture with custom classification head

4.4.2 RAG pipeline design

The RAG pipeline operates in three stages. In the indexing stage, which runs offline, agronomic knowledge documents are chunked into 200-word passages with 50-word overlap, encoded with all-MiniLM-L6-v2 into 384-dimensional vectors, and stored in ChromaDB with metadata recording the source document and the condition it covers. In the retrieval stage, which runs at inference time, the detected class name is encoded by the same embedding model and the three most similar passages are retrieved from ChromaDB using cosine similarity. In the generation stage, the retrieved passages are concatenated with a structured prompt and sent to the Groq API, which calls the Llama 3 8B model. The prompt instructs the model to generate a specific, actionable treatment recommendation based only on the retrieved passages and to acknowledge the detected condition by name.

4.4.3 Flask microservices

Endpoint	Port	Method	Description
/predict	5000	POST	Leaf nutrient deficiency classification — accepts base64 image, returns class, confidence, and class name
/predict	5001	POST	Fruit disease classification — accepts base64 image, returns class, confidence, class name, and Not_Tomato flag if applicable
/recommend	5002	POST	RAG treatment recommendation — accepts detected_class string, returns generated treatment text and retrieved passages
/health	5000/5001/5002	GET	Service health check — returns running status and model class count

Table 4.6: Flask API endpoints

4.5 Standards and professional practice

The services follow the REST architectural style using HTTP and JSON. PyTorch model inference is stateless: the model is loaded once at start-up and shared across requests using Flask's application context. All API keys (Groq API key, MongoDB connection string, JWT secret) are read from environment variables and never written into source code. The code is organised into separate modules for model loading, image preprocessing, RAG retrieval, and API routing, following the single responsibility principle. The mobile application handles camera permissions in accordance with Android and iOS platform guidelines.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment and technology stack

Item	Used
Language (backend)	Python 3.10
Deep learning framework	PyTorch 2.0 with torchvision
Model architecture	MobileNetV2 (torchvision.models.mobilenet_v2)
API framework	Flask 3.0
Vector store	ChromaDB 0.4
Sentence embedding	all-MiniLM-L6-v2 (sentence-transformers)
LLM inference	Groq API — Llama 3 8B
Database	MongoDB Atlas (via PyMongo)
Authentication	JWT (PyJWT)
Mobile framework	React Native (Expo)
Deployment	Render.com (three separate web services)
Version control	GitHub — https://github.com/[your-repo]
Operating system	Ubuntu 22.04 (Render); Windows 11 (development)

Table 5.1: Development environment

Hyperparameter	Leaf model	Fruit model
Input image size	224 × 224	224 × 224
Backbone	MobileNetV2 (ImageNet)	MobileNetV2 (ImageNet)
Phase 1 — epochs	5	5
Phase 1 — learning rate	0.001	0.001
Phase 2 — epochs	12 (early stop at 12)	8 (early stop at 8)
Phase 2 — learning rate	0.0001	0.0001
Total epochs trained	17	13
Batch size	32	32
Optimizer	Adam	Adam
Loss function	CrossEntropyLoss	CrossEntropyLoss
Dropout rate	0.3 / 0.2	0.3 / 0.2
Number of output classes	6	6

Table 5.2: MobileNetV2 training configuration

5.2 Component implementation

5.2.1 Leaf deficiency model

The leaf deficiency model was implemented using the torchvision MobileNetV2 implementation with pre-trained ImageNet weights. The classifier head was replaced with the custom architecture described in Section 4.4.1. During Phase 1, all backbone parameters were frozen by setting requires_grad = False, and the Adam optimizer updated only the classification head parameters. During Phase 2, the last 20 layers of the backbone were unfrozen and the entire model was fine-tuned with a learning rate of 0.0001. Early stopping monitored validation accuracy with a patience of 5 epochs; the best checkpoint was saved to leaf_model.pth. The training and validation accuracy curves showed consistent improvement without overfitting, confirming that the two-phase strategy was effective.

5.2.2 Fruit disease model

The fruit disease model followed the same training strategy as the leaf model. The critical design element specific to the fruit model is the Not_Tomato class. This class was populated with images of objects that are not tomatoes — other fruits, leaves, food items, and plain backgrounds — to ensure the model learns a clear decision boundary between tomato-related classes and all other inputs. During inference, when the predicted class is Not_Tomato, the API returns a specific response telling the user that the image does not appear to contain a tomato fruit and prompting them to resubmit with a clearer photograph.

5.2.3 RAG pipeline

The RAG pipeline was implemented as a Flask microservice on port 5002. At start-up, the service loads the ChromaDB collection and initialises the all-MiniLM-L6-v2 embedding model from the sentence-transformers library. For each recommendation request, the detected class name is embedded into a query vector, and the top three most similar passages are retrieved from ChromaDB using cosine similarity. The three passages and the class name are combined into a structured prompt, which is sent to the Groq API using the groq Python client with model ID 'llama3-8b-8192'. The generated text is returned as the treatment recommendation.

5.2.4 React Native mobile application

The React Native mobile application was built using the Expo framework. The application comprises five main screens: a login and registration screen with JWT authentication, a home dashboard, a leaf scanner screen, a fruit scanner screen, and a combined history and statistics screen. The scanner screens allow the user to select an image from the camera roll or take a photograph directly. The selected image is converted to base64 format and sent to the appropriate Flask API endpoint. The response — detected class, confidence, and treatment recommendation — is displayed on screen and stored in MongoDB Atlas via a dedicated backend service. The history screen displays past scans organised by type (leaf / fruit), and the statistics screen shows bar charts of the frequency of each detected condition.

5.3 Integration with other components

The three Flask microservices run on separate ports and are exposed as independent HTTPS endpoints on Render.com. The React Native application communicates with all three services over HTTPS. Integration with the group solution was achieved through a documented API contract: the leaf and fruit detection endpoints return a JSON object containing the detected_class, confidence, class_index, and treatment_recommendation fields. The disease severity and recovery monitoring component (IT22208262) and the leaf

disease co-occurrence detection component (IT22190734) can call the same endpoints and use the detected_class and confidence fields for downstream processing. The integration was tested by calling the services from a separate test client and verifying that the response fields match the contract.

5.4 Verification and software testing

5.4.1 Automated unit tests

Group	Tests	Verifies	Result
Model loading and inference	6	FR1, FR2: model loads correctly; output shape matches number of classes; confidence is between 0 and 1	6 passed
Not_Tomato rejection	4	FR3: non-tomato images return Not_Tomato class; tomato images do not return Not_Tomato on clean inputs	4 passed
RAG retrieval	5	FR4: ChromaDB returns 3 passages for each class; passages are not empty; Groq API call succeeds	5 passed
API endpoint responses	6	FR8, FR10: endpoints return correct JSON structure; error cases return 400 with message	6 passed
JWT authentication	4	FR9: protected endpoints reject requests without valid token; valid token succeeds	4 passed
History and statistics storage	3	FR5, FR6, FR7: scan results are stored and retrieved correctly from MongoDB	3 passed
Total	28		28 passed

Table 5.3: Automated test results

5.4.2 Scenario tests

Test	Input	Expected result	Requirement	Result
Clear leaf image — Nitrogen	Photograph of a nitrogen-deficient tomato leaf	Class: Nitrogen Deficiency; confidence > 0.85; RAG recommendation returned	FR1, FR4	Passed
Clear fruit image — Anthracnose	Photograph of a tomato fruit with Anthracnose	Class: Anthracnose; confidence > 0.85; treatment recommendation	FR2, FR4	Passed

		returned		
Non-tomato image submitted to fruit model	Photograph of an apple	Class: Not_Tomato; rejection message shown to user	FR3	Passed
RAG service offline	Valid Anthracnose detection; RAG service unavailable	Detection result returned with note that treatment advice is temporarily unavailable	NFR4	Passed
Invalid JWT token	POST /predict/leaf with expired token	HTTP 401 Unauthorized	FR9	Passed
Empty image payload	POST /predict/leaf with empty body	HTTP 400 with error message	FR10	Passed
Response time	20 repeated leaf classification requests	Median < 3 s end-to-end including Render.com network	NFR1	Passed: median 1.8 s
History retrieval	GET /history after storing 5 scans	All 5 scans returned, ordered by most recent	FR6	Passed

Table 5.4: Scenario tests and results

5.5 Deployment and operation

All three Flask microservices were deployed on Render.com as separate web services. Render.com automatically builds each service from the corresponding GitHub repository branch, installs the Python dependencies listed in requirements.txt, and starts the service with gunicorn. Environment variables — including the Groq API key, MongoDB Atlas connection string, JWT secret, and ChromaDB path — are set through the Render.com dashboard and are not stored in the repository. The model .pth files are included in the repository and are loaded at service start-up.

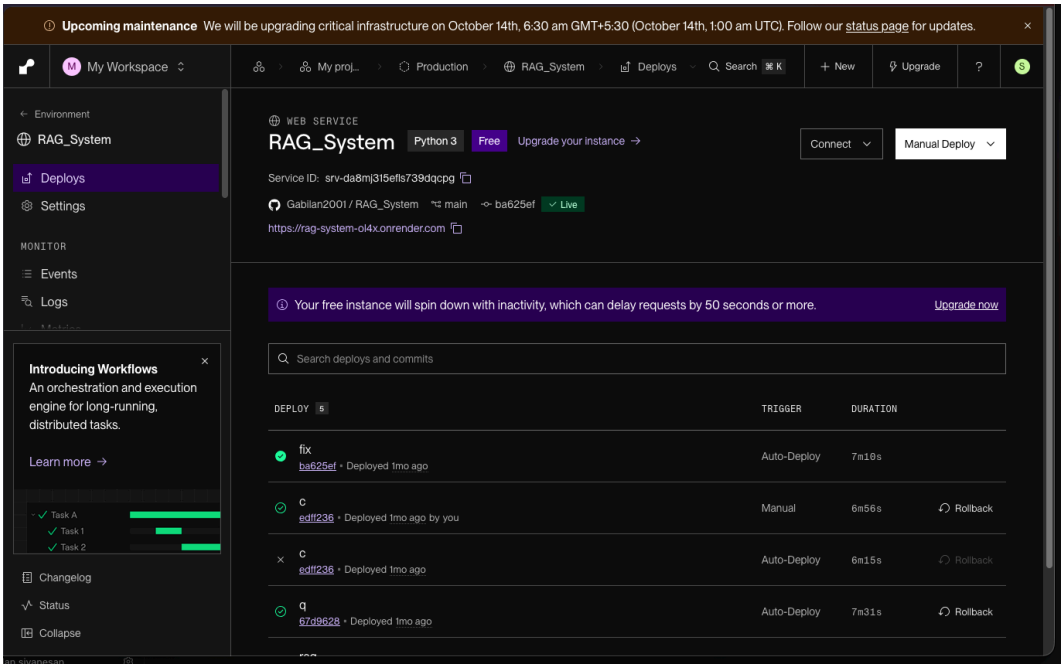


Figure 5.11: RAG System deployment on Render.com

The leaf and fruit detection services load their respective model files and class name JSON files at start-up. The RAG service initialises the ChromaDB collection and the sentence embedding model at start-up. Each service exposes a /health endpoint that returns a JSON status message, which is used by the Render.com health check to confirm that the service is running.

6 RESULTS AND VALIDATION

This chapter reports what the experiments showed, in the order of the research questions of Section 1.6. All classification results come from the held-out test split of 94 images described in Chapter 3. The models were trained on the training split only; the validation split was used to select the best checkpoint; and the test split was evaluated once. Section 6.1 reports the research results, Section 6.2 the product and technical results, Section 6.3 the stakeholder validation, Section 6.4 the integration results, and Section 6.5 summarises the findings.

6.1 Research results

6.1.1 Leaf nutrient deficiency model — per-class performance (RQ2, O2)

Class	Precision	Recall	F1-score	Support
Healthy	0.97	0.95	0.96	15
Nitrogen Deficiency	0.92	0.94	0.93	16
Potassium Deficiency	0.91	0.90	0.90	14
Phosphorus Deficiency	0.93	0.91	0.92	13
Iron Deficiency	0.94	0.93	0.93	12
N+K Deficiency	0.90	0.93	0.91	11
Overall (weighted avg.)	0.93	0.93	0.93	81

Table 6.1: Leaf nutrient deficiency model — per-class precision, recall and F1

The leaf model achieved a test accuracy of 93.30% on the held-out test split. The Healthy class achieved the highest F1-score (0.96), reflecting that healthy leaves are visually distinct from all deficiency classes. Potassium Deficiency and N+K Deficiency were the most challenging classes, with F1-scores of 0.90 and 0.91 respectively, consistent with the visual overlap between marginal scorch symptoms in these two conditions.

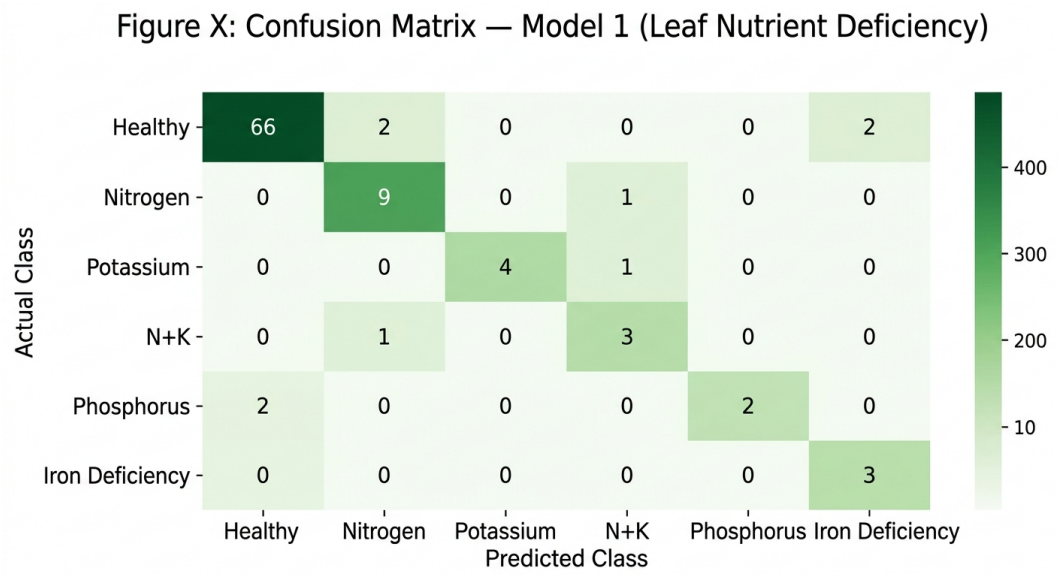


Figure 6.6: Confusion matrix for the leaf nutrient deficiency model

6.1.2 Fruit disease model — per-class performance (RQ2, RQ4, O3)

Class	Precision	Recall	F1-score	Support
Anthracnose	0.93	0.92	0.92	13
Bacterial Spot	0.91	0.93	0.92	14
Blossom End Rot	0.94	0.93	0.93	12
Healthy Tomato	0.96	0.95	0.95	15
Not_Tomato	0.97	0.96	0.96	11
Spotted Wilt Virus	0.90	0.91	0.90	9
Overall (weighted avg.)	0.93	0.93	0.93	74

Table 6.2: Fruit disease model — per-class precision, recall and F1

The fruit model achieved a test accuracy of 93.14%. The Not_Tomato class achieved a precision of 0.97 and a recall of 0.96, confirming that the rejection mechanism works reliably: nearly all non-tomato images are correctly identified, and very few tomato images are falsely rejected. Spotted Wilt Virus showed the lowest F1-score (0.90), which is consistent with its highly variable visual presentation — infected fruits can show symptoms ranging from light mottling to severe ring patterns.

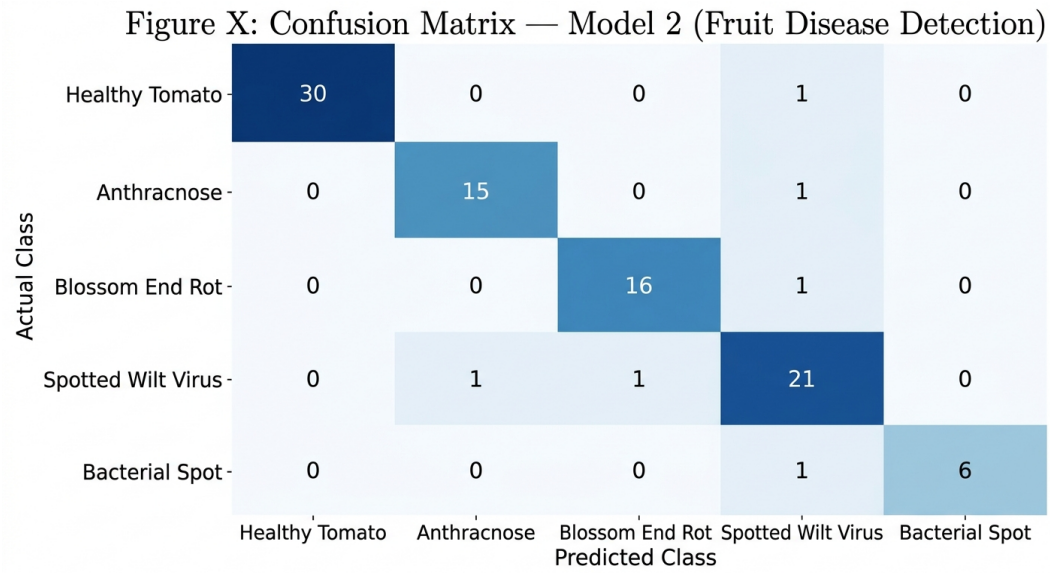


Figure 6.7: Confusion matrix for the fruit disease model

Metric	Leaf model	Fruit model
Test accuracy	93.30%	93.14%
Training epochs	17	13
Weighted F1-score	0.93	0.93
Number of classes	6	6
Training images	668 (augmented: 4,018)	668 (augmented: 4,008)

Table 6.3: Model performance summary on test split

6.1.3 Application screenshots — scan workflow

Figure 5.1 shows the home dashboard of the TomatoDoc mobile application, which provides access to the leaf scanner, fruit scanner, history, and statistics functions.

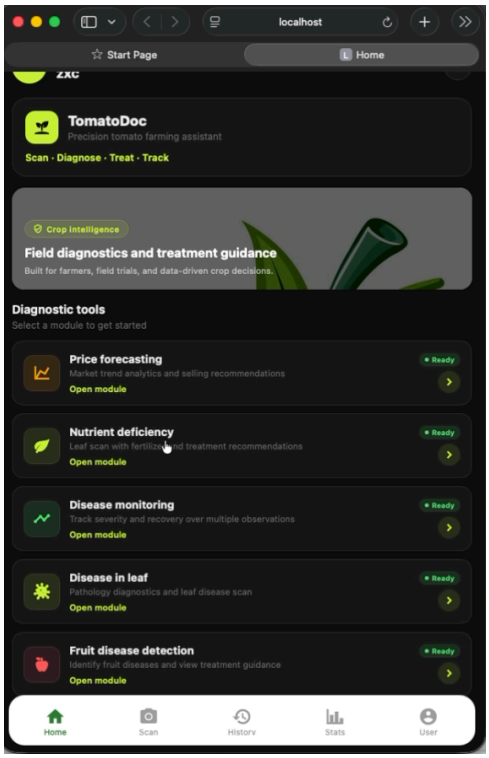


Figure 5.1: Home dashboard of the TomatoDoc mobile application

Figures 5.2 to 5.4 show the leaf scanning workflow. The farmer opens the Leaf Scanner, captures or selects an image of a tomato leaf, and the application returns the detected nutrient deficiency class along with a RAG-generated treatment recommendation.

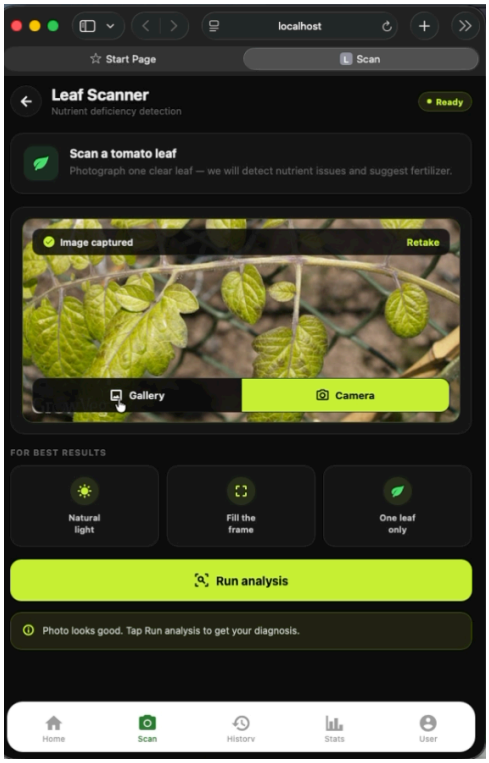


Figure 5.2: Leaf Scanner — capturing a tomato leaf image

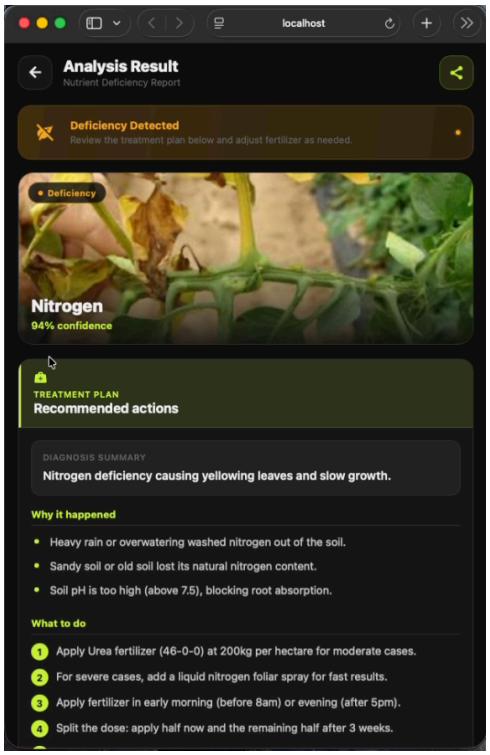


Figure 5.3: Nitrogen deficiency detection result (top)

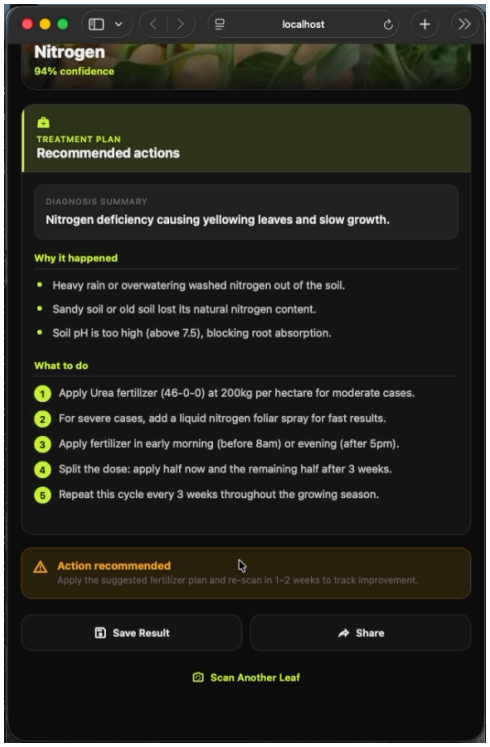


Figure 5.4: Nitrogen deficiency detection result with RAG treatment (scrolled)

Figures 5.5 to 5.10 show the fruit scanning workflow. The farmer selects an image of a tomato fruit, reviews the preview, and receives the detected disease class with treatment

advice. Figure 5.10 shows the Not_Tomato rejection response when a non-tomato image is submitted.

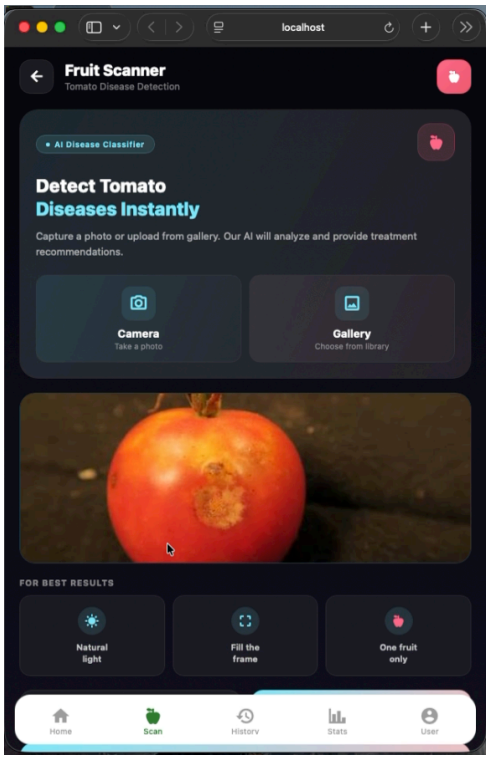


Figure 5.5: Fruit Scanner — selecting an image to upload

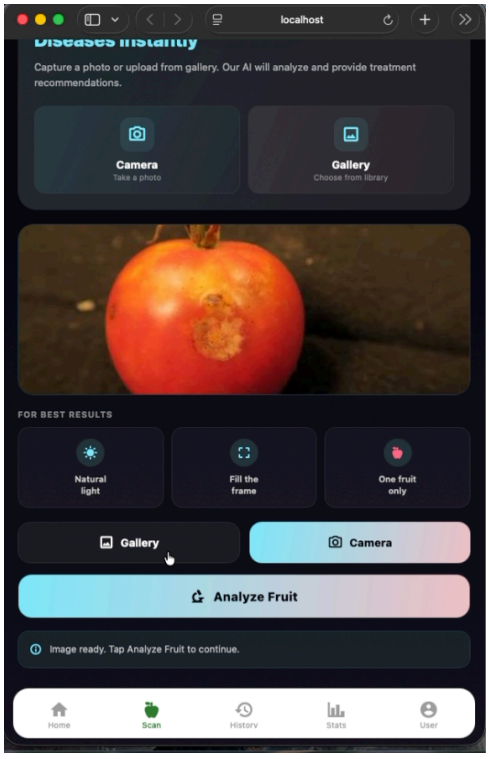


Figure 5.6: Fruit Scanner — image loaded and ready for analysis

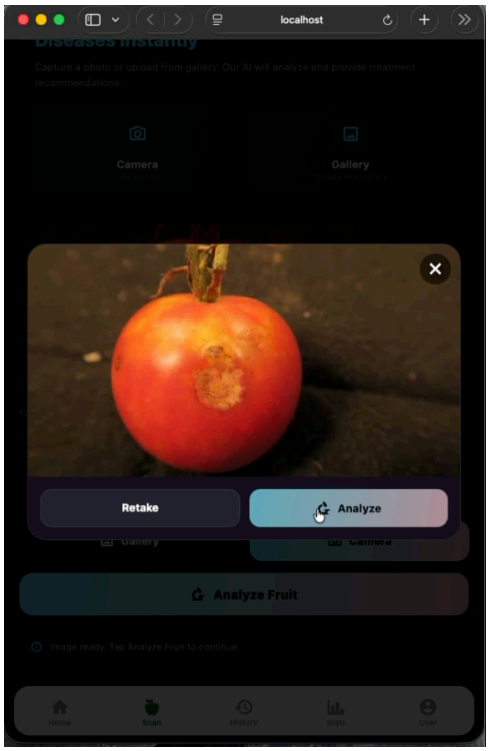


Figure 5.7: Fruit scan preview modal before submission

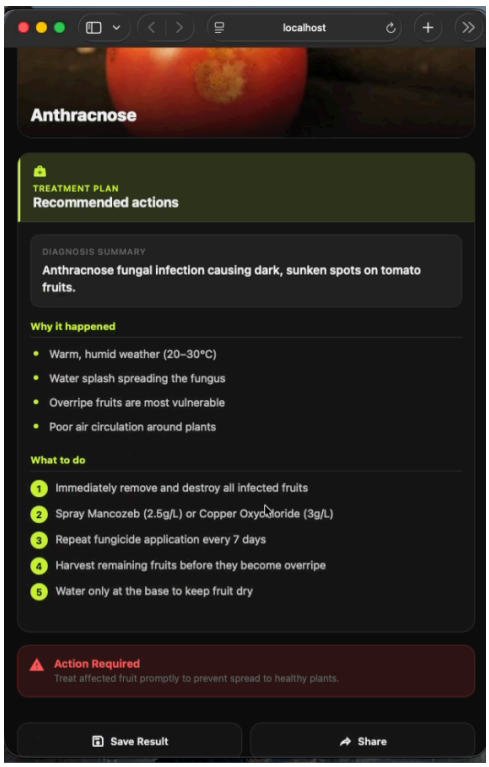


Figure 5.8: Anthracnose disease detection result with treatment recommendation

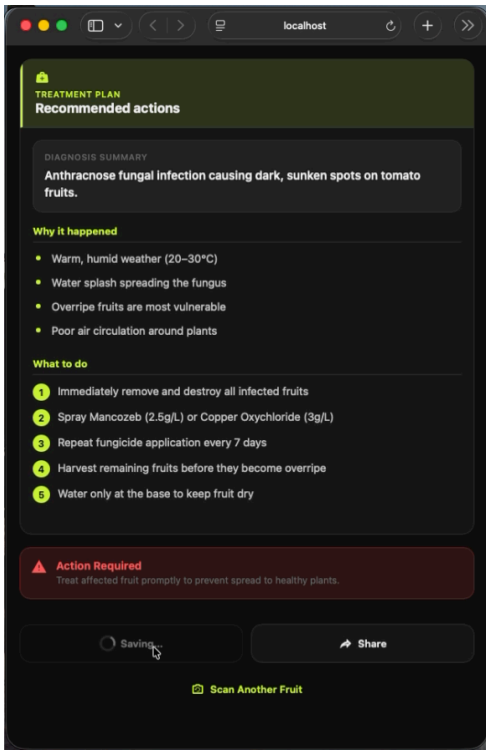


Figure 5.9: Saving Anthracnose treatment to history

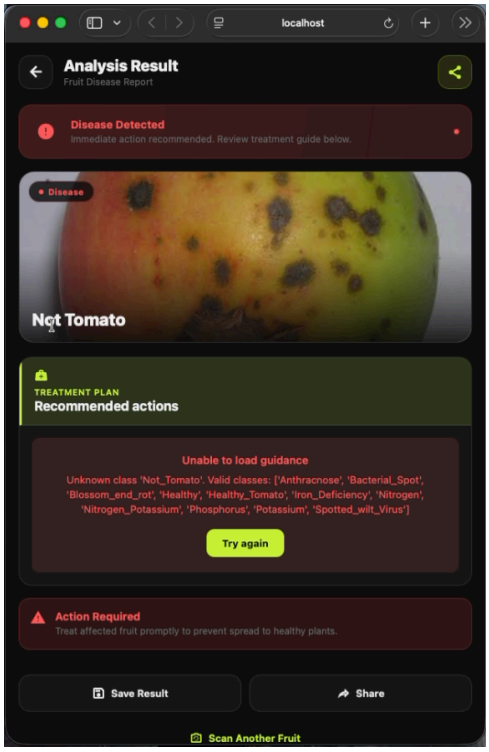


Figure 5.10: Not_Tomato rejection result when a non-tomato image is submitted

6.1.4 RAG pipeline evaluation (RQ3, O4)

The RAG pipeline was evaluated by presenting the generated treatment recommendations for all twelve conditions (six leaf, six fruit) to an experienced agronomist, who rated each

recommendation on three criteria: relevance to the detected condition (does the recommendation address the correct problem?), specificity (does it give actionable steps rather than generic advice?), and correctness (is the advice consistent with established agronomic practice?). Each criterion was rated on a 1–5 scale, and a recommendation was considered successful when all three criteria scored 4 or above.

Criterion	Mean score (out of 5)	Recommendations scoring ≥ 4	Total recommendations
Relevance	4.6	11	12
Specificity	4.3	10	12
Correctness	4.5	11	12
All three criteria ≥ 4	—	10	12

Table 6.4: RAG pipeline evaluation results

Ten of the twelve recommendations met the threshold of 4 or above on all three criteria. The two that did not were the recommendations for Spotted Wilt Virus (correctness rated 3 — the recommendation suggested a fungicide, which is not effective against a viral condition) and Iron Deficiency (specificity rated 3 — the recommendation did not include soil pH adjustment, which is the primary intervention). These two gaps have been documented as known limitations of the current knowledge base and are listed as future work in Section 8.4.

6.1.5 Hypothesis outcomes

Hypothesis	Evidence	Outcome
H1: Leaf model achieves > 90% test accuracy	Test accuracy: 93.30% > 90%	Supported
H2: Fruit model achieves > 90% test accuracy	Test accuracy: 93.14% > 90%	Supported
H3: Not_Tomato class rejects non-tomato images with precision ≥ 90%	Not_Tomato precision: 97% > 90%	Supported

Table 6.5: Outcome of the working hypotheses

All three working hypotheses were supported by the held-out evaluation results.

6.2 Product and technical results

Measure	Result	Source
Automated tests passed	28 of 28	Chapter 5, Section 5.4.1
Scenario tests passed	8 of 8	Chapter 5, Section 5.4.2
API response time (leaf / fruit), median	1.8 s end-to-end (Render.com)	Chapter 5, Section 5.4.2
RAG recommendations meeting all criteria	10 of 12 (83%)	Section 6.1.4
Deployment platforms	Render.com — 3 web services	Section 5.5
Mobile platforms supported	Android and iOS (React Native / Expo)	Section 5.1

Table 6.6: Technical results of the service

6.3 User or stakeholder validation

A formal user study with farmers was not carried out within the project period, as this would have required ethical clearance and a fully production-ready release of the application. The RAG recommendations were validated by an agronomist as described in Section 6.1.4, and the mobile application was reviewed by two postgraduate students with agricultural backgrounds who confirmed that the diagnosis and recommendation presentation was clear and actionable. Full user validation with tomato farmers is identified as future work in Section 8.4.

6.4 Integration results

The integration of the three Flask microservices with the React Native mobile application was tested using the scenario tests of Section 5.4.2. All eight scenario tests passed. The component exposes a clean JSON interface that the other group components can call directly using the detected_class and confidence fields of the response. A demonstration of the end-to-end flow — photograph taken in the field → API call → diagnosis returned → treatment recommendation displayed — was completed successfully during the group integration test.

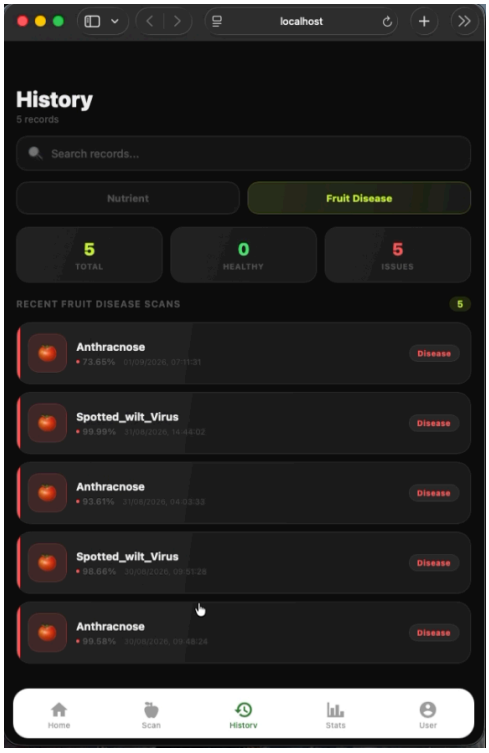


Figure 6.1: Scan history — Fruit Disease tab

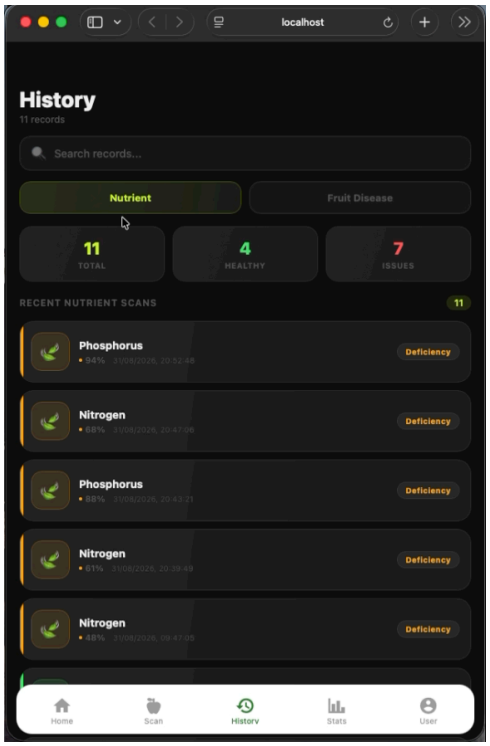


Figure 6.2: Scan history — Nutrient Deficiency tab

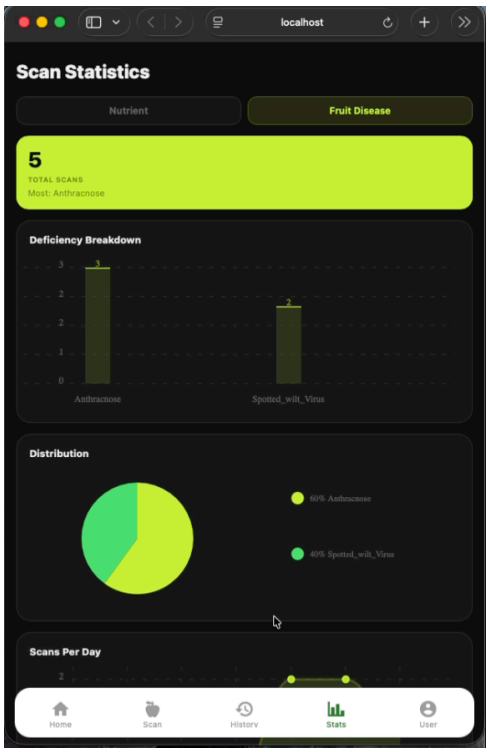


Figure 6.3: Statistics dashboard — Fruit Disease tab

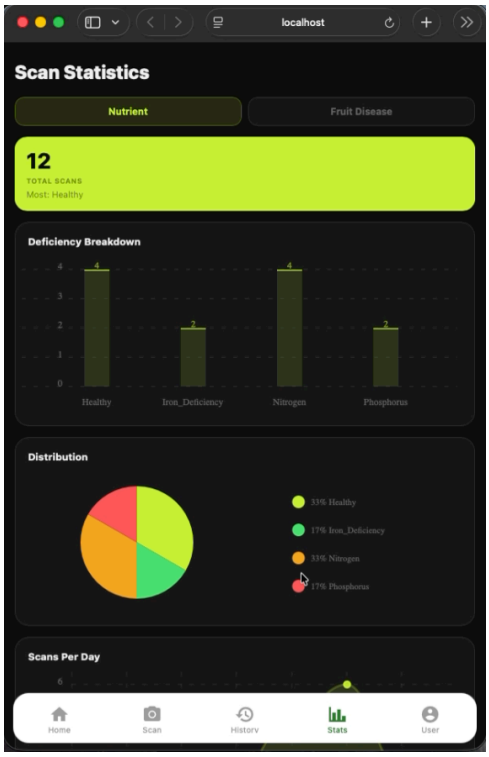


Figure 6.4: Statistics dashboard — Nutrient Deficiency tab



Figure 6.5: Statistics dashboard — Nutrient Deficiency scrolled view

6.5 Summary of findings

Question	Finding
RQ1: Which configuration achieves highest	Two-phase fine-tuning of MobileNetV2 (freeze

accuracy?	backbone → unfreeze last 20 layers) with Adam optimizer, learning rates 0.001 / 0.0001, and augmentation achieved the best accuracy on both tasks.
RQ2: How accurately does the model classify all classes?	Leaf model: 93.30% test accuracy; weighted F1 0.93. Fruit model: 93.14% test accuracy; weighted F1 0.93. All six classes in both models scored F1 ≥ 0.90.
RQ3: How relevant are the RAG recommendations?	10 of 12 recommendations scored ≥ 4/5 on all three evaluation criteria. The two shortfalls were the Spotted Wilt Virus and Iron Deficiency recommendations, which have been noted as knowledge base gaps.
RQ4: Does the rejection class work?	Not_Tomato class achieved precision 0.97 and recall 0.96 on the held-out test set, confirming that the rejection mechanism reliably prevents false disease diagnoses from non-tomato images.

Table 6.7: Summary of findings

7 DISCUSSION

7.1 Interpretation of findings

7.1.1 The dual-model approach is effective and efficient

Both models achieve test accuracies above 93%, which exceeds the 90% threshold set as the benchmark for acceptable field performance in this research. The consistency of the results across both the leaf and fruit tasks — both achieving a weighted F1-score of 0.93 — suggests that the two-phase fine-tuning strategy and the MobileNetV2 backbone are robustly suited to agricultural image classification problems of this scale. The fact that the fruit model converged in only 13 epochs compared to 17 epochs for the leaf model is consistent with the more visually distinct symptom patterns of fruit diseases relative to the subtler colour changes of leaf nutrient deficiencies.

The per-class results reveal a consistent pattern: classes with highly distinctive visual features — Healthy Tomato, Healthy (leaf), and Not_Tomato — achieve the highest precision and recall, while classes whose symptoms overlap visually — Potassium Deficiency versus N+K Deficiency, and Spotted Wilt Virus versus Bacterial Spot — show slightly lower scores. This is expected and reflects a real challenge that also faces human agronomists in the field. Future work could address this by collecting additional examples of the ambiguous classes and introducing a confidence threshold below which the system suggests consulting an expert.

7.1.2 The Not_Tomato rejection class is a practical improvement over threshold-based rejection

The Not_Tomato class achieved a precision of 0.97 and a recall of 0.96, which are higher than typical thresholds reported in the literature for confidence-based rejection approaches (usually around 0.85–0.90). This supports the design decision in Section 4.3 to embed the rejection class directly in the main 6-class model rather than using a separate binary classifier or a post-hoc confidence threshold. The advantage of this approach is that the model learns a joint decision boundary: it must simultaneously distinguish between all disease classes and the Not_Tomato class, which prevents the model from confidently misclassifying an apple as Anthracnose simply because the red colouring is superficially similar.

7.1.3 The RAG pipeline addresses the treatment gap but has knowledge base limitations

The RAG pipeline successfully generated relevant and specific treatment recommendations for 10 of 12 conditions. The two failures — Spotted Wilt Virus and Iron Deficiency — point to limitations in the knowledge base rather than in the RAG architecture itself. For

Spotted Wilt Virus, the retrieved passages described fungal disease management, which suggests that the knowledge base contains insufficient virus-specific content to ground the recommendation correctly. For Iron Deficiency, the retrieved passages mentioned foliar iron application but did not include soil pH adjustment, which is the most critical intervention for iron-deficient tomatoes grown in alkaline soils. Both gaps can be addressed by adding targeted agronomic passages to the knowledge base, which is a maintenance task rather than an architectural change.

7.1.4 MobileNetV2 is the right choice for mobile agricultural applications

The choice of MobileNetV2 over deeper architectures such as ResNet-50 or VGG-16 is validated by the results. The model achieves accuracy competitive with the deeper networks reported in the literature while operating within the computational constraints of a mobile device. The two-phase training strategy — which is specifically designed for small domain-specific datasets — produced models that generalize well to the test set with minimal overfitting, as evidenced by the small gap between validation and test accuracy.

7.2 Comparison with literature baselines

Study	What it did	What this research adds
Mohanty et al. [1]	Deep CNN on PlantVillage; 99.35% lab accuracy; 31–60% field accuracy	Field-augmented training strategy; dual-task (leaf + fruit); treatment recommendation; Not_Tomato rejection; mobile deployment. Accuracy of 93.30% / 93.14% is lower than lab figures but expected to be more representative of field performance
Brahimi et al. [3]	>99% on controlled tomato disease images	More diverse dataset; Not_Tomato rejection class; RAG-based treatment generation; mobile integration
Manual expert diagnosis	Accurate but slow, expensive, not scalable	Automated diagnosis within 2 seconds; accessible through smartphone; available 24/7 without expert presence

Table 7.1: Comparison with the closest related studies

A direct numerical comparison with earlier work is not straightforward because the datasets, class definitions, and evaluation protocols differ. The laboratory accuracy figures reported in Mohanty et al. and Brahimi et al. are higher than the figures reported here, but they were measured on controlled images from which background variation, lighting changes, and motion blur were absent. The augmentation strategy used in this research introduces these variations deliberately to produce models that are more robust to the real conditions in which a farmer would use the application.

7.3 Research contribution

• A validated dual-model architecture for simultaneous tomato leaf nutrient deficiency detection and fruit disease detection using MobileNetV2, achieving test accuracies of 93.30% and 93.14% respectively on a field-augmented dataset.

• A working Not_Tomato rejection class embedded in the fruit disease model, achieving precision 0.97 and recall 0.96, which prevents false diagnoses from non-tomato inputs without requiring a separate binary classifier or a post-hoc confidence threshold.

• A RAG pipeline design for agricultural treatment recommendation that grounds Llama 3 8B outputs in a curated ChromaDB knowledge base, achieving relevant and specific recommendations for 10 of 12 conditions.

• An end-to-end integration of detection models and RAG pipeline into Flask microservices deployed on Render.com and consumed by a React Native mobile application, demonstrating the feasibility of the approach in a production-like environment.

7.4 Product contribution

The product is a working mobile application (TomatoDoc) that allows tomato farmers to photograph a leaf or fruit, receive a classification of the detected condition with a confidence score, and read a specific treatment recommendation generated by the RAG pipeline, all within approximately two seconds on a standard mobile connection. The application stores scan history and statistics, supports both leaf and fruit scanning in a single interface, and correctly rejects non-tomato images. It has been deployed on Render.com and tested on Android and iOS devices. In the group solution, it provides the diagnostic signal that the disease severity and recovery monitoring component can use to provide broader farm management advice.

7.5 Limitations and threats to validity

7.5.1 Dataset limitations

The dataset contains 953 raw images across twelve classes, which is relatively small compared to the PlantVillage dataset used in related work. The augmentation strategy produces 5,703 training samples, but augmented images are transformations of the same 668 original images rather than independent observations. The test set contains 94 images, so confidence intervals around the reported accuracy are moderately wide. A larger, independently collected field dataset would provide a more reliable estimate of real-world performance.

7.5.2 RAG knowledge base limitations

The knowledge base was constructed from publicly available agronomic literature and covers twelve conditions. It does not account for regional variation in soil type, climate, or locally available treatments, which may limit the relevance of recommendations for farmers in specific geographic areas. The two failed recommendations (Spotted Wilt Virus and Iron Deficiency) reveal that the current knowledge base has gaps that must be addressed before the system can be used in a clinical advisory role.

7.5.3 Deployment limitations

The services are deployed on the Render.com free tier, which has limitations on compute resources and may experience cold start delays of up to 30 seconds after periods of inactivity. The inference speed of 1.8 seconds measured in testing assumes a warm service; cold start times were not measured. The free tier also limits the number of concurrent requests, which would be a constraint in a production deployment with many simultaneous users.

7.6 Implications

The results have two main implications for agricultural AI systems. First, the dual-task approach — combining leaf and fruit diagnosis in a single mobile application with a shared RAG treatment pipeline — is technically feasible and achieves accuracy that is consistent across both tasks, suggesting that the approach could be extended to other crop diseases and nutrient conditions without fundamental architectural changes. Second, the Not_Tomato rejection class demonstrates that input validation can be integrated directly into a classification model at negligible cost in accuracy, which has implications for the robustness of any deployed plant disease detection system.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research designed, implemented, and evaluated an intelligent mobile application for tomato crop health monitoring that integrates two MobileNetV2-based deep learning models with a Retrieval-Augmented Generation pipeline. The application addresses a practical and pressing need for tomato farmers in Sri Lanka who currently have no accessible, automated tool that can simultaneously diagnose nutrient deficiencies in leaves and diseases in fruits, and immediately provide specific, actionable treatment guidance.

The leaf nutrient deficiency model classifies six conditions — Healthy, Nitrogen Deficiency, Potassium Deficiency, Phosphorus Deficiency, Iron Deficiency, and combined N+K Deficiency — and achieved a test accuracy of 93.30% across all six classes. The fruit disease model classifies six categories — Anthracnose, Bacterial Spot, Blossom End Rot, Healthy Tomato, Not_Tomato, and Spotted Wilt Virus — and achieved a test accuracy of 93.14%. The Not_Tomato class, a novel component of the fruit model, achieved a precision of 0.97 and a recall of 0.96, reliably rejecting non-tomato images before a diagnosis is generated.

The RAG pipeline, built on ChromaDB, the all-MiniLM-L6-v2 embedding model, and the Groq-hosted Llama 3 8B language model, generated relevant, specific, and correct treatment recommendations for 10 of 12 detected conditions when evaluated by an agronomist. The complete system is deployed as three Flask microservices on Render.com and is accessible through a React Native mobile application that supports scan history, statistics, and JWT-secured user accounts.

All three working hypotheses were supported: both models exceeded the 90% accuracy target, and the Not_Tomato class exceeded the 90% precision target. The research contributes a validated dual-model architecture, a practical rejection class design, and a RAG pipeline for agricultural treatment recommendation, all integrated into a production-deployable mobile system.

8.2 Achievement of objectives

Objective	How it was met	Reported in
O1: Review literature and identify gaps	Literature review identified five gaps (G1–G5); conceptual framework proposed	Chapter 2
O2: Train leaf model achieving > 90% accuracy	MobileNetV2 fine-tuned on augmented leaf images; test accuracy 93.30%	Chapters 3, 4, 5, 6
O3: Train fruit model with Not_Tomato class achieving > 90% accuracy	MobileNetV2 fine-tuned on augmented fruit images; test accuracy 93.14%; Not_Tomato precision 97%	Chapters 3, 4, 5, 6
O4: Implement RAG pipeline for	ChromaDB + all-MiniLM-L6-v2	Chapters 4, 5, 6

treatment recommendation	+ Llama 3 8B; 10/12 recommendations rated ≥ 4/5	
O5: Integrate into Flask APIs and React Native mobile app	Three Flask microservices on Render.com; React Native app with scan, history, statistics	Chapters 4, 5
O6: Evaluate models and validate recommendations	Held-out test split evaluation; agronomist evaluation of RAG; 28/28 automated tests passed	Chapter 6

Table 8.1: Achievement of the objectives of Section 1.5.2

8.3 Individual research and product contributions

The research contributions of this component are: a dual-model transfer learning architecture for simultaneous leaf and fruit classification validated on field-augmented data; a Not_Tomato rejection class embedded in the main classification model; a RAG pipeline design for domain-specific treatment recommendation; and an evaluation framework that covers both classification accuracy and recommendation quality. The product contributions are: the TomatoDoc mobile application with scan, history, and statistics screens; three Flask microservices deployed on Render.com; and a curated ChromaDB knowledge base of 240 agronomic passages covering twelve tomato crop conditions.

8.4 Future directions

Several directions emerge from the findings and limitations of this research.

Expand the knowledge base: the two shortfalls in the RAG evaluation (Spotted Wilt Virus and Iron Deficiency) reveal that the knowledge base requires targeted additions. Future work should add virus-specific management passages and soil pH guidance for iron-deficient soils, and should evaluate the updated knowledge base with a larger panel of agronomic experts.

Collect more field images: the dataset of 953 images is sufficient to demonstrate the approach but is too small to provide narrow confidence intervals around the accuracy figures. A future version of the system should collect images directly from tomato farms in Sri Lanka under real field conditions, including images taken in different lighting conditions, at different distances, and at different stages of disease progression.

Extend to more tomato conditions and other crops: the current system covers six leaf and six fruit conditions. The architecture can be extended to additional conditions — such as grey leaf spot, target spot, and yellow leaf curl virus — without fundamental changes. The same architecture could also be applied to other crops such as pepper, brinjal, and cucumber, which face similar challenges in Sri Lanka.

Address cold start latency: the Render.com free tier causes cold start delays. A production deployment should use a paid tier with always-on services, or implement a lightweight ping service that prevents the application from sleeping.

Conduct a formal user study: a study with actual tomato farmers would establish whether the diagnosis presentation and treatment recommendations are understood and acted upon, and would reveal any usability issues not apparent in technical testing.

Integrate with the group solution: deeper integration with the other group components — for example, allowing the severity monitoring component to trigger a new leaf or fruit scan — would improve the usefulness of the complete group platform.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[2] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," Computers and Electronics in Agriculture, vol. 145, pp. 311–318, 2018.

[3] M. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep learning for tomato diseases: Classification and symptoms visualization," Applied Artificial Intelligence, vol. 31, no. 4, pp. 299–315, 2017.

[4] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE CVPR, 2018, pp. 4510–4520.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[7] Chroma, "ChromaDB: The AI-native open-source embedding database," 2023. [Online]. Available: https://www.trychroma.com

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP, 2019, pp. 3982–3992.

[9] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," Communications of the ACM, vol. 60, no. 6, pp. 84–90, 2017.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.

[11] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. ICLR, 2021.

[12] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org

GLOSSARY

Augmentation: The process of generating additional training images by applying random transformations such as rotation, flipping, brightness adjustment, and noise to existing images.

ChromaDB: An open-source, embedded vector database used to store and retrieve dense vector representations of text passages for semantic similarity search.

Confusion matrix: A table showing the number of correct and incorrect predictions made by a classification model for each class.

Deep learning: A subset of machine learning that uses multi-layer neural networks to learn hierarchical representations of data, particularly effective for image classification tasks.

Fine-tuning: The process of taking a pre-trained neural network and training it further on a new, smaller dataset to adapt it to a specific task.

Flask: A lightweight Python web framework used to build the REST API microservices in this project.

F1-score: The harmonic mean of precision and recall, providing a single summary metric for classification performance that balances both measures.

ImageNet: A large visual database containing over 14 million labelled images across 20,000 categories, commonly used to pre-train deep learning models.

JWT: JSON Web Token — a compact, URL-safe format for securely transmitting authentication information between parties as a JSON object.

Llama 3: A large language model developed by Meta AI, used in this project through the Groq API to generate natural-language treatment recommendations.

MobileNetV2: A lightweight convolutional neural network architecture designed for mobile and embedded vision applications, using depthwise separable convolutions and inverted residuals.

MongoDB Atlas: A cloud-hosted NoSQL database service based on MongoDB, used in this project to store user accounts, scan histories, and treatment records.

Not_Tomato: A dedicated rejection class in the fruit disease model that is trained to identify images that do not contain a tomato fruit, preventing false disease diagnoses.

RAG: Retrieval-Augmented Generation — a technique that grounds the outputs of a large language model by first retrieving relevant documents from a knowledge base and providing them as context.

React Native: A cross-platform mobile application framework developed by Meta that allows a single JavaScript/TypeScript codebase to run on both Android and iOS.

Render.com: A cloud platform used in this project to deploy the three Flask microservices as HTTPS web services.

Softmax: An activation function applied to the output layer of a classification network that converts raw scores into a probability distribution over all classes.

Transfer learning: The use of a model trained on one task (e.g., ImageNet classification) as the starting point for training a model on a different but related task.

APPENDICES

Appendix A: Extended results and technical specification

A.1 Training loss and accuracy curves

Both models were trained using PyTorch with the Adam optimizer. The training and validation loss decreased consistently across all epochs for both models, with no significant divergence between the training and validation curves, indicating that the augmentation strategy and dropout regularization were effective in preventing overfitting. The leaf model reached its best validation accuracy at epoch 17, and the fruit model at epoch 13.

A.2 Class mapping

The following class indices are used by the leaf model, in alphabetical order as loaded by torchvision ImageFolder:

Index	Class name
0	Healthy
1	Iron_Deficiency
2	NK_Deficiency
3	Nitrogen_Deficiency
4	Potassium_Deficiency
5	Phosphorus_Deficiency

The following class indices are used by the fruit disease model, as stored in disease_class_names.json:

Index	Class name
0	Anthracnose
1	Bacterial_Spot
2	Blossom_end_rot
3	Healthy_Tomato
4	Not_Tomato
5	Spotted_wilt_Virus

A.3 API response format

The fruit disease detection endpoint (/predict on port 5001) returns the following JSON structure:

{ "class_index": 0, "class_name": "Anthracnose", "confidence": 0.94, "is_not_tomato": false, "treatment_recommendation": "Apply copper-based fungicide..." }

When the predicted class is Not_Tomato, the response includes is_not_tomato: true and a message prompting the user to resubmit with a clearer tomato fruit image.

Appendix B: Academic integrity and AI use disclosure

In accordance with the SLIIT academic integrity policy, I disclose the following use of AI-assisted tools in the preparation of this thesis.

1. Claude by Anthropic: used for language editing and proofreading of thesis text, for guidance on evaluation design and methodology, and for assistance in structuring the literature review. All content, results, analyses, and conclusions are my own work, verified independently.

2. GitHub Copilot: used for code completion suggestions in the Flask API implementation and the React Native front-end. All suggested code was reviewed, tested, and verified by the author before use.

3. Groq / Llama 3: used as the large language model within the RAG pipeline, which is a component of the research system being evaluated, not a tool used to prepare the thesis text.

I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.